

Day 2 Learning

#THE STATIC KEYWORD IN JAVA

The static keyword in Java is a non-access modifier used mainly for memory management. It signifies that a member belongs to the class itself rather than to a specific instance or object of the class.

1. STATIC VARIABLES OR CLASS VARIABLES

- When a variable is declared as static, a single copy of the variable is created and shared among all objects of the class. Memory allocation for static variables happens only once when the class is loaded in memory.
- It is used for properties that are common to all objects, such as a company name or college name.
- It makes the program memory efficient because it does not create a new copy for every object.

2. STATIC METHODS

- A static method belongs to the class rather than the object.
- It can be invoked without creating an instance of the class.
- It can access static data members and change their values.
- Restrictions: A static method cannot use non-static data members or call non-static methods directly. It cannot use this or super keywords.

3. STATIC BLOCKS

- A static block is used for initializing static variables.
- It is executed exactly once when the class is first loaded into memory by the Java ClassLoader.
- It executes before the main method and before any constructors.

4. STATIC INNER CLASSES

- In Java, you can define a class within another class. If the inner class is marked as static:
- It does not need a reference to the outer class instance.
- It can only access static members of the outer class.
- It is often used to group classes that are only used by the outer class to increase encapsulation.

HOW STATIC MEMBERS ARE INITIALIZED

Static members are initialized when the class is loaded. The order of execution is:

1. Static variables are assigned their default or explicit values.
2. Static blocks are executed in the order they appear in the code. This happens before any objects are created and before the main method starts.

#COMMON INTERVIEW QUESTIONS ON STATIC KEYWORD:

1. Why is the main method static in Java? Answer: It is static so that the JVM can invoke it without creating an instance of the class. This saves memory and allows the program to start execution immediately.
2. Can we override static methods? Answer: No, we cannot override static methods. If you declare a static method with the same signature in a child class, it is called Method Hiding, not overriding.
3. Can a static method access non-static variables? Answer: No, a static method cannot access non-static or instance variables directly because the static method exists at the class level, whereas non-static variables only exist when an object is created.
4. What is the difference between a static block and a constructor? Answer: A static block runs once when the class is loaded, while a constructor runs every time a new object or instance of the class is created.
5. Can we have static methods in an interface? Answer: Yes, since Java 8, interfaces can have static methods which contain implementation details. Prior to Java 8, all variables in interfaces were implicitly static and final.
6. What happens if you remove the static keyword from the main method? Answer: The program will compile, but it will throw a `NoSuchMethodError` or `Main method not found` at runtime because the JVM specifically looks for a static main method to start the application.

THIS KEYWORD IN JAVA:

'this' keyword is a reference variable in Java that refers to the current object. It is primarily used to avoid confusion between class attributes and parameters with the same name.

1. **TO REFER TO CURRENT CLASS INSTANCE VARIABLES:** This is the most common use. When a method or constructor has parameters that have the same name as the instance variables, the 'this' keyword is used to distinguish the instance variable from the local variable.
2. **TO INVOKE CURRENT CLASS METHOD:** You can invoke the method of the current class by using the 'this' keyword. If you do not use it, the compiler automatically adds it while invoking the method.
3. **TO INVOKE CURRENT CLASS CONSTRUCTOR:** The `this()` constructor call can be used to invoke the current class constructor. This process is known as constructor chaining. It is used to reuse the constructor code. Note that `this()` must be the first statement in the constructor.
4. **TO PASS AS AN ARGUMENT IN THE METHOD:** The 'this' keyword can also be passed as an argument in the method call. It is mainly used in event handling or when we need to pass the current object to another method.
5. **TO RETURN THE CURRENT CLASS INSTANCE:** You can return the 'this' keyword as a statement from the method. In such a case, the return type of the method must be the class type.

SUPER KEYWORD IN JAVA

The super keyword in Java is a reference variable which is used to refer to the immediate parent class object. Whenever you create an instance of a subclass, an instance of the parent class is created implicitly, which is referred to by the super reference variable.

1. **TO REFER TO IMMEDIATE PARENT CLASS INSTANCE VARIABLE:** If a parent class and a child class have the same field names, the super keyword can be used to access the data field or variable of the parent class.
2. **TO INVOKE IMMEDIATE PARENT CLASS METHOD:** It is used to call the methods of the parent class. This is especially useful in Method Overriding, where the child class wants to call the parent class version of the overridden method.
3. **TO INVOKE IMMEDIATE PARENT CLASS CONSTRUCTOR:** The super() keyword is used to invoke the immediate parent class constructor.
 - It must be the first statement in the subclass constructor.
 - If you do not manually call super(), the compiler automatically provides an implicit call to the no-argument constructor of the parent class.

#COMMON INTERVIEW QUESTIONS ON THIS AND SUPER

1. Can this() and super() both be used in the same constructor?

Answer: No. Both this() and super() must be the first statement in a constructor. Since you can only have one first statement, they cannot be used together in the same constructor.

2. What is the main difference between this and super?

Answer: This keyword refers to the current class instance, while the super keyword refers to the immediate parent class instance.

3. Why must super() or this() be the first statement in a constructor?

Answer: Java ensures that the parent class is initialized before the child class. By making it the first statement, the object is built from the base up, ensuring all inherited members are available for use in the child constructor.

4. What happens if a parent class does not have a default constructor and the child class does not call super(parameters)?

Answer: The code will fail to compile. If the parent class only has a parameterized constructor, the child class must explicitly call that constructor using super(args) in its own constructor.

5. Can you use this and super in a static method?

Answer: No. Both this and super refer to objects (instances). Since static methods belong to the class and not to any object, these keywords cannot be used inside static methods.

6. What is constructor chaining?

Answer: Constructor chaining is the process of calling one constructor from another constructor within the same class (using `this()`) or from a parent class (using `super()`).

#ABSTRACT CLASSES AND INTERFACES IN JAVA:

Abstraction is the process of hiding the implementation details and showing only the functionality to the user. In Java, abstraction is achieved using Abstract Classes and Interfaces.

1. ABSTRACT CLASS:

- An abstract class is a class that is declared with the `abstract` keyword. It can have both abstract methods (methods without a body) and non-abstract methods (methods with a body).
- It cannot be instantiated, meaning you cannot create an object of an abstract class.
- It can have constructors and static methods.
- It can have final methods which force the subclass not to change the body of the method.
- It can have member variables that are not final and static.
- The constructor of abstract class should be protected, because these constructors are only be called by child classes which extend the abstract class, so there is no sense of making them public.

2. INTERFACE:

- An interface in Java is a blueprint of a class. It is a collection of abstract methods and static constants.
- It is used to achieve fully 100 percent abstraction and multiple inheritance in Java.
- All methods in an interface are implicitly public and abstract (before Java 8).
- All variables in an interface are implicitly public, static, and final.
- Since Java 8, interfaces can have default and static methods with implementation.
- Since Java 9, interfaces can have private methods.

KEY DIFFERENCES

- **Type of methods:** Abstract classes can have both abstract and concrete methods. Interfaces (mostly) have abstract methods.
- **Final Variables:** Variables declared in a Java interface are final by default. An abstract class may contain non-final variables.
- **Type of variables:** Abstract class can have static, non-static, final, and non-final variables. The interface only has static and final variables.
- **Implementation:** An abstract class can provide the implementation of an interface. An interface cannot provide the implementation of an abstract class.
- **Inheritance:** A class can extend only one abstract class but can implement multiple interfaces.

INTERVIEW QUESTIONS

1. Can an abstract class be declared as final?

Answer: No. The purpose of an abstract class is to be inherited by a subclass. If you make it final, it cannot be inherited, which creates a compile-time error.

2. Can we have an abstract method in a non-abstract class?

Answer: No. If a class contains even one abstract method, the entire class must be declared as abstract.

3. Can an interface implement another interface?

Answer: No, an interface cannot implement another interface. However, an interface can extend another interface (or multiple interfaces) using the extends keyword.

4. What is a Marker Interface?

Answer: A marker interface is an interface that has no methods or fields (e.g., Serializable, Cloneable). It is used to provide "metadata" to the JVM so that the JVM knows to treat the object in a specific way.

5. If a class implements two interfaces that have a method with the same name and signature, which one is executed?

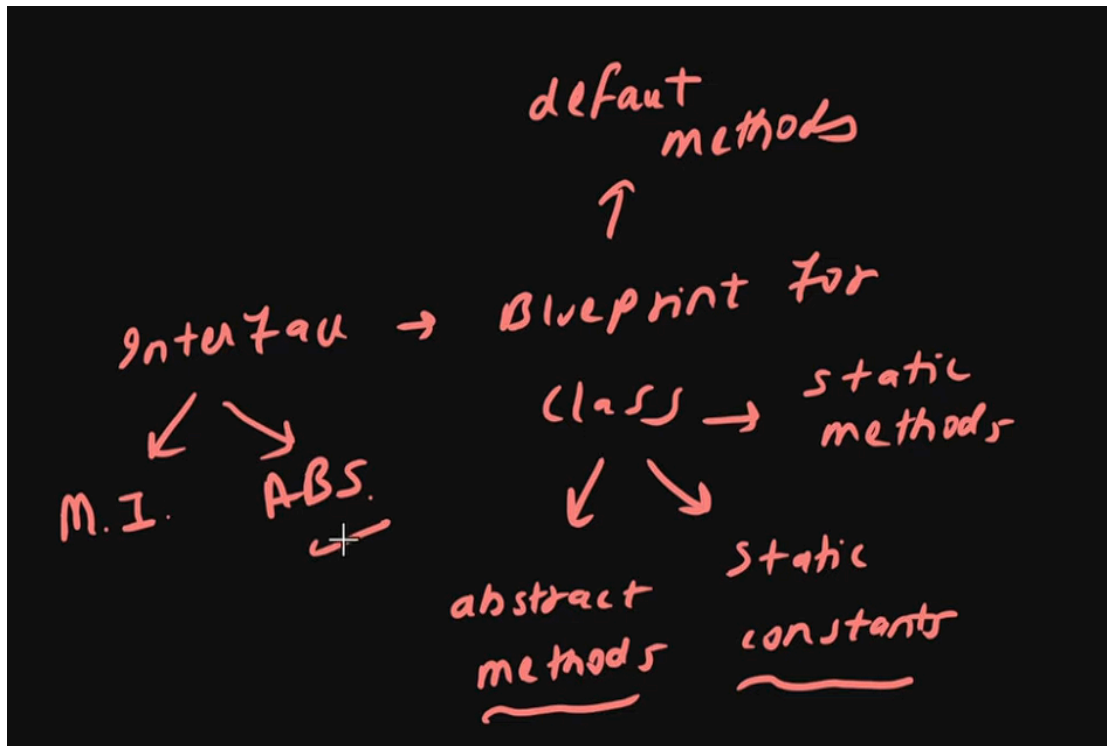
Answer: If the methods are abstract, the class provides one single implementation for both. However, if both interfaces have a "default" method with the same signature, it creates a conflict. The programmer must resolve this by overriding the method in the implementation class and using the "super" keyword to specify which interface method to call.

6. Can we create an object of an abstract class using the new keyword?

Answer: No, you cannot directly instantiate an abstract class. However, you can create an instance of a subclass that extends the abstract class, or you can use an anonymous inner class to provide immediate implementation.

7. Why does Java not support multiple inheritance with classes but supports it with interfaces?

Answer: Multiple inheritance with classes leads to the "Diamond Problem" (ambiguity in which method to inherit). Interfaces avoid this because, traditionally, they did not contain method implementations. Even with default methods, Java forces the developer to manually resolve any naming conflicts, thus removing ambiguity.



#ENCAPSULATION IN JAVA

Encapsulation is one of the four fundamental concepts of Object-Oriented Programming (OOP). It is the technique of wrapping data (variables) and code acting on the data (methods) together as a single unit. In encapsulation, the variables of a class are hidden from other classes and can be accessed only through the methods of their current class.

1. HOW TO ACHIEVE ENCAPSULATION??

To achieve encapsulation in Java, you must follow two main steps:

- Declare the variables of a class as private.
- Provide public setter and getter methods to modify and view the variables values.

2. DATA HIDING:

- Encapsulation is often referred to as data hiding because it isolates the data from the direct access of other classes. Users can only interact with the data through a controlled interface (the public methods).

3. BENEFITS OF ENCAPSULATION:

- Better Control: You can make a class read-only (by providing only getters) or write-only (by providing only setters).
- Security: You can add validation logic inside the setter methods to ensure that only valid data is stored in the variables.
- Flexibility: You can change one part of the code without affecting other parts. The internal implementation can change as long as the public methods remain the same.

#QUESTIONS ON ENCAPSULATION

1. What is the difference between Encapsulation and Abstraction?

Answer: Encapsulation is about hiding the internal state and grouping data and behavior, whereas Abstraction is about hiding the complexity and showing only the essential features. Encapsulation is a way to implement abstraction.

2. Can a class be 100 percent encapsulated?

Answer: Yes, if all the data members of the class are private and we provide public getter and setter methods for all of them, the class is considered fully encapsulated. This is often called a Java Bean class.

3. What is a Getter and a Setter?

Answer: A Getter is a method used to retrieve the value of a private variable. A Setter is a method used to update or set the value of a private variable.

4. Is encapsulation related to security?

Answer: Yes, it provides a basic level of security by preventing unauthorized classes from changing sensitive data directly. By using setters, you can add checks or authentication before allowing a value to be changed.

5. Can we have a private constructor in an encapsulated class?

Answer: Yes. A private constructor prevents the class from being instantiated from outside. This is often used in the Singleton Design Pattern, but it does not stop the class from being encapsulated.

6. Why is Encapsulation also called a Data Hide or Black Box?

Answer: It is called a black box because the internal workings and data of the object are hidden from the outside world. The user only knows how to use the public methods without needing to know how they work internally.